

Divide and Conquer

If a problem can be divided into smaller problems such that, the solution of smaller problems gives us the solution of bigger problem.

DIVIDE - Divide larger problem into smaller problem.

CONQUER - Solve each smaller problem until they are a base case.

COMBINE (Optional) - Combine solution of smaller problems to find solution of larger problem.

Backtracking

Build solution step by step.

At each step, we discard steps(s) that do not result in a solution.

→ Maze.

→ Sudoku

→ 8 queen problem.

→ knight tour.

Multiply two $\downarrow$ numbers, without using * operator.
positive

$$\text{multiply}(x, y) = \begin{cases} x, & \text{if } y = 1 \\ y, & \text{if } x = 1 \\ 0, & \text{if } x = 0 \text{ or } y = 0 \\ x + \text{multiply}(x, y-1), & \text{otherwise.} \end{cases}$$

$$x * y = \underbrace{x + x + \dots + x}_{y \text{ times}}$$

$\Downarrow$

$$x + \underbrace{(x + x + \dots + x)}_{(y-1) \text{ times}} \Rightarrow x * (y-1)$$

Assignment

① Divide two numbers and give quotient.

② Divide two numbers and give remainder.

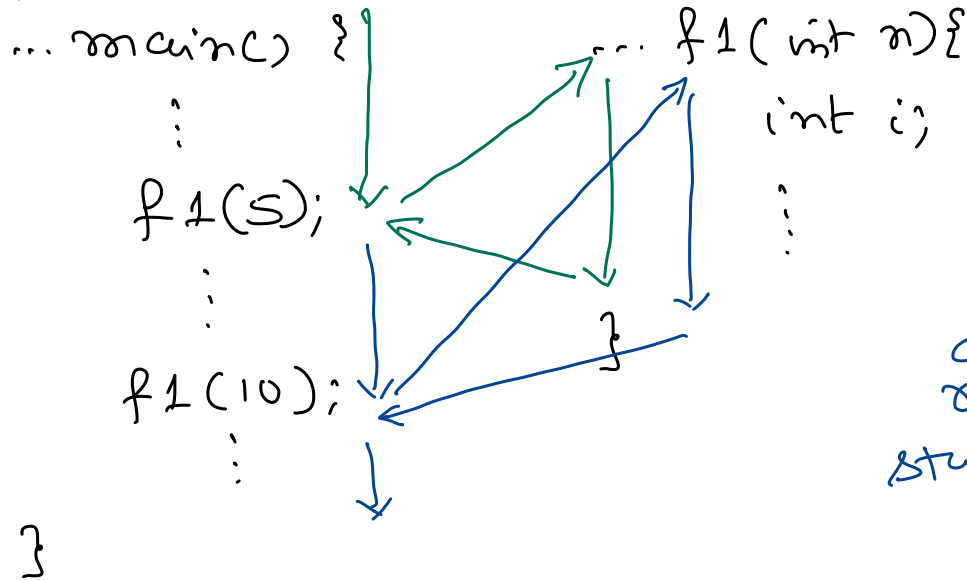
multiply (2, 10927)

positive

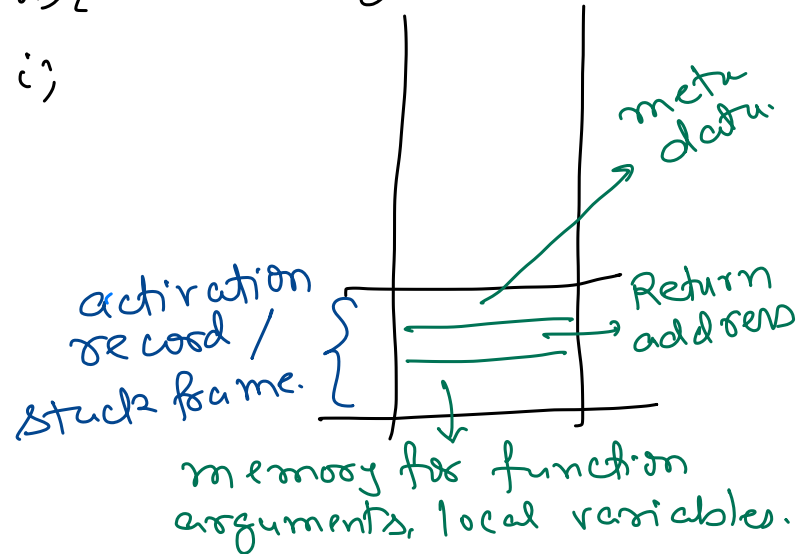
Assignment: Multiply two $\wedge$ numbers optionally.

Assignment: Multiply two Integers (-ve and +ve).

How do function call works?



Call Stack/
System Stack.



```

int factorial(int n) {
    if ((n==0) || (n==1))
        return 1;
    return n * factorial(n-1);
}

```

factorial(3) $\rightarrow$ A1

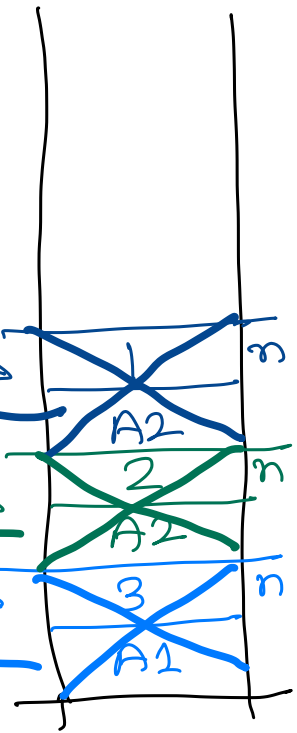
Stack unwinding

$\rightarrow$ Remove stack frame when function call is over.

1.
A2

2

6



CPU Registers AX BX CX DX } general purpose registers.

Calling convention

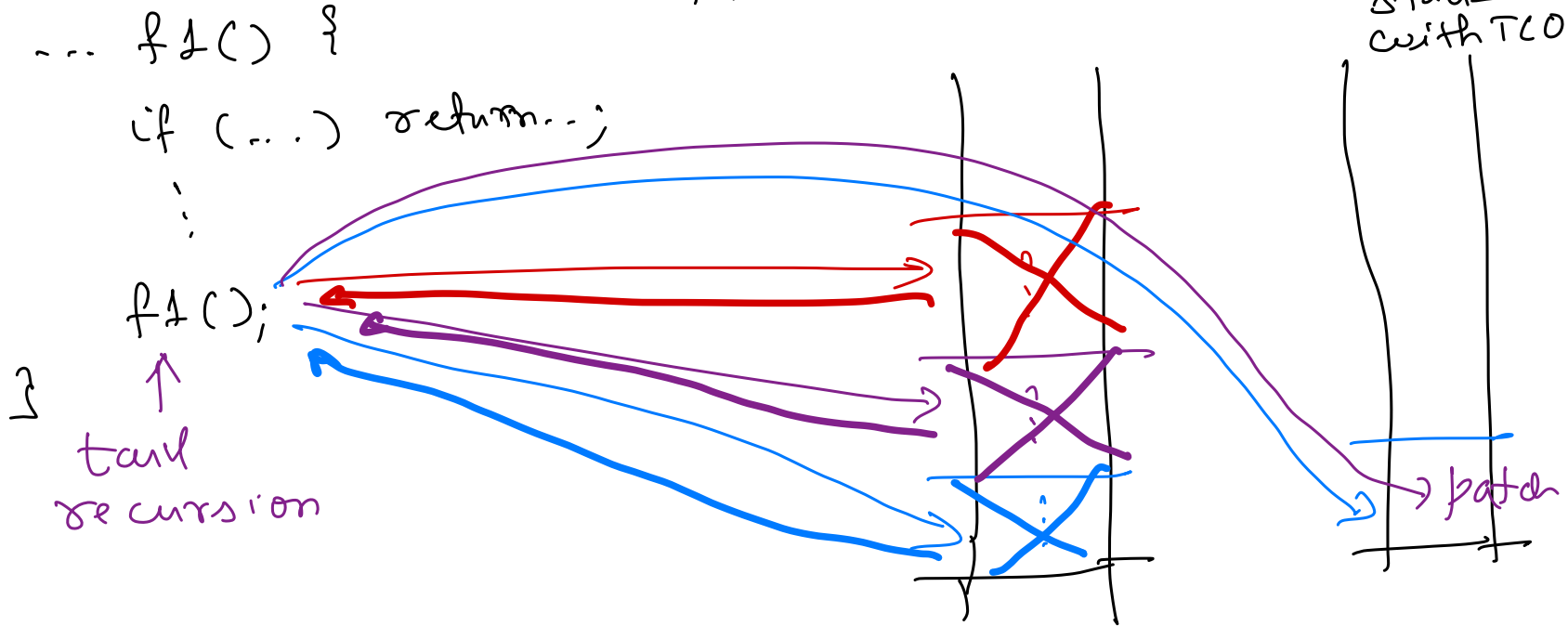
→ in which order arguments will be pushed on stack.

$f1(2, 3, 4)$



→ who clears stack frame after function call is over

Tail Recursion → when last statement in function is the recursive call.



Tail Call Optimisation (TCO) → Shares/reuses the stack frame for tail recursive calls.

Tree

→ Non-linear data structure.

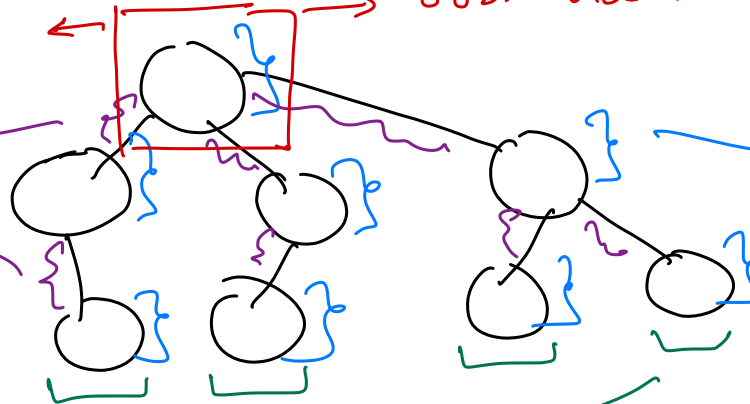
→ Tree is a collection of nodes.

→ Tree store parent - child relationship.

root node do not have any parent. → root node ⇒ top most node of tree.

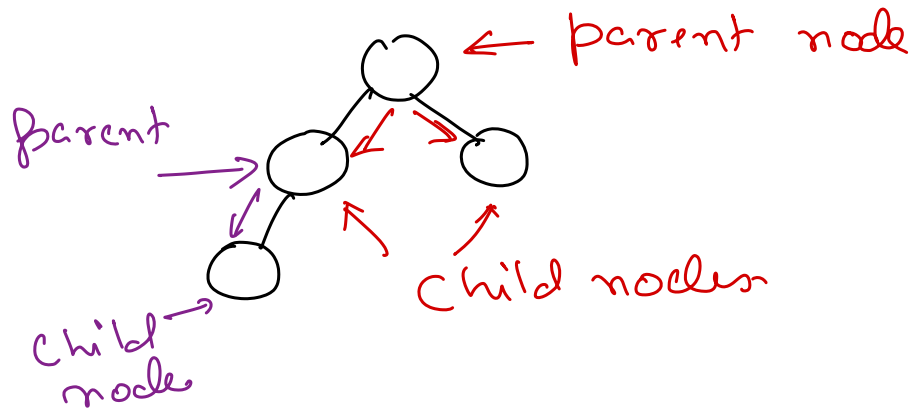
⇓
Each tree will have at most one root node.

edges



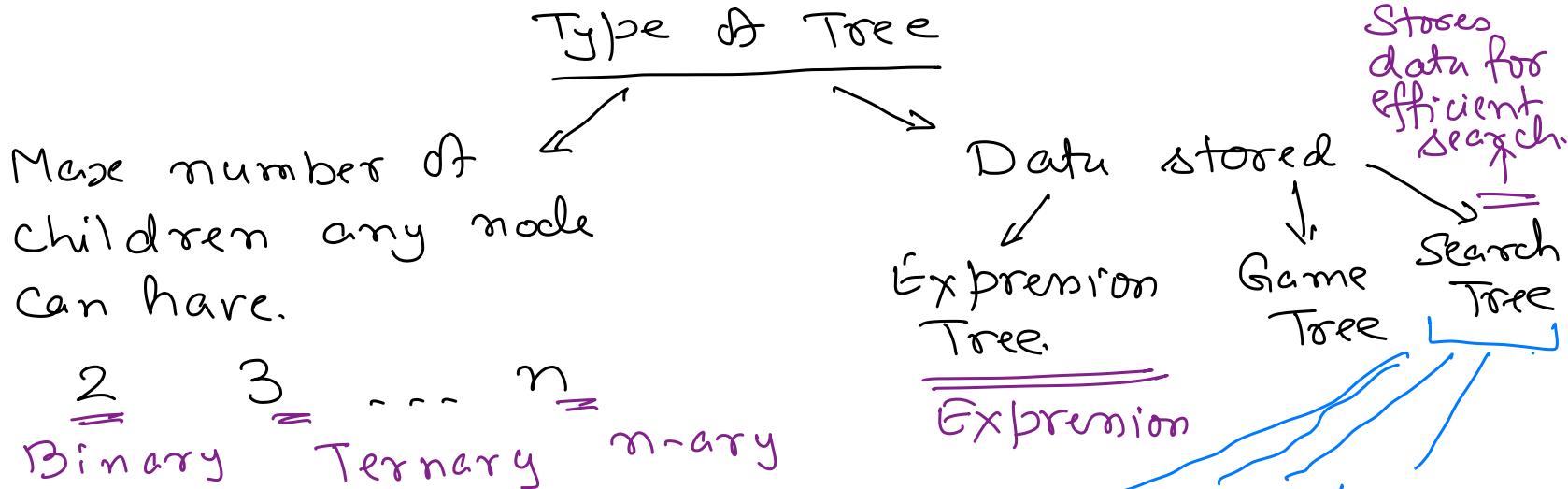
Nodes

Leaf nodes ⇒ nodes that do not have any children.



Each parent node
keeps track of
child nodes.

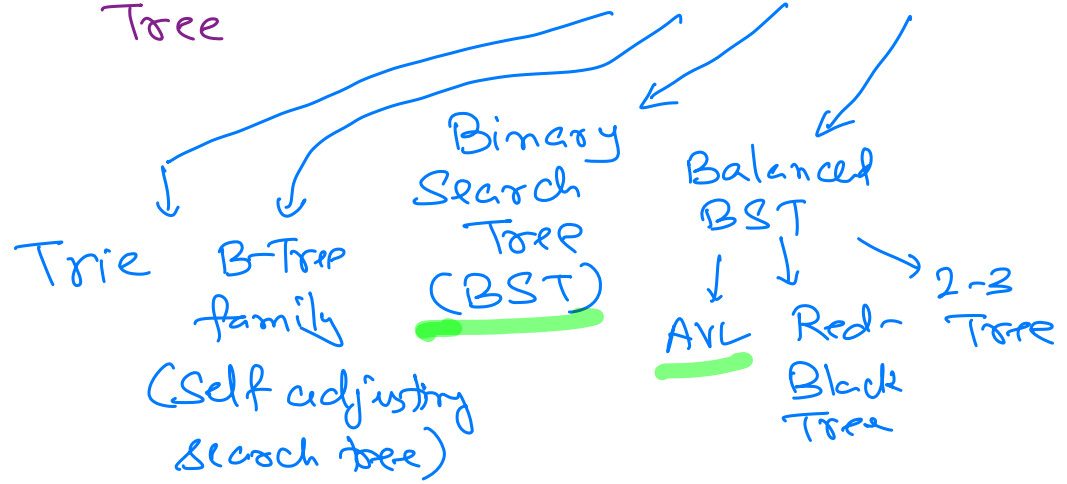
child nodes do
not track their
parent nodes.



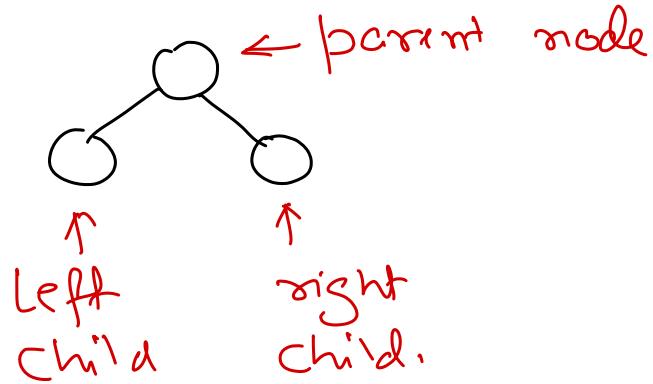
Tree

Tree

Tree



Binary Tree



Binary Tree Traversals

Depth first

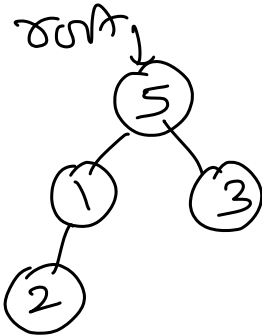
Breadth first

↓
Level Order

(a) (c) (b) → Inorder

(c) (a) (b) → Pre order

(a) (b) (c) → Post order



$\text{traverse}(\text{root}) =$
depth first

root → empty

done, if root is empty.

(a) traverse (root's left child)

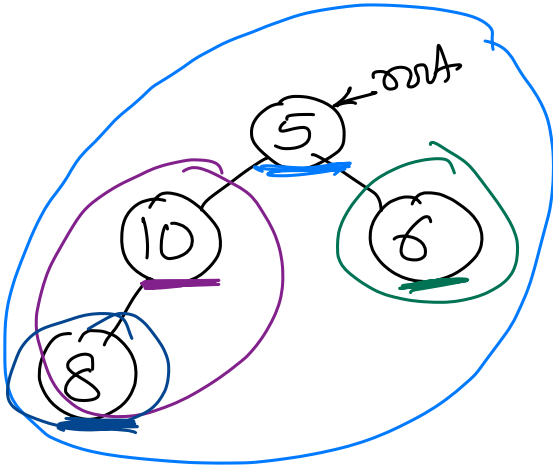
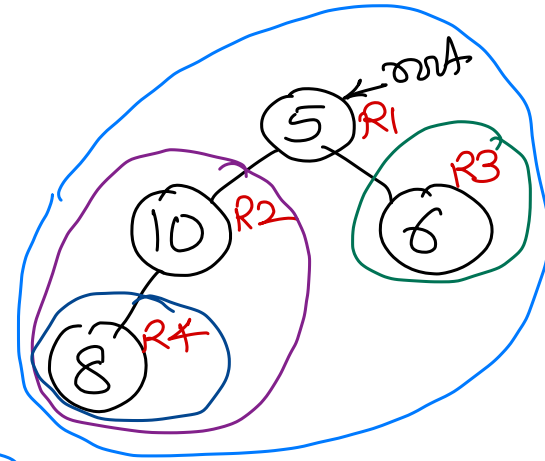
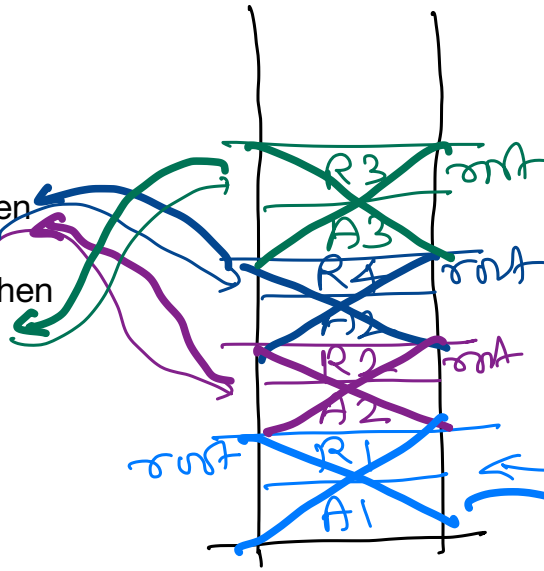
(b) traverse (root's right child)

(c) process root node.

throw error

PreOrder(root)

- if (root is empty) then
 - Stop.
- Process root node's data.
- If (root node's left child exists) then
 - PreOrder(root's left child). **A2**
- If (root node's right child exists) then
 - PreOrder(root's right child) **A3**
- Stop.



O/P:

5 10 8 6

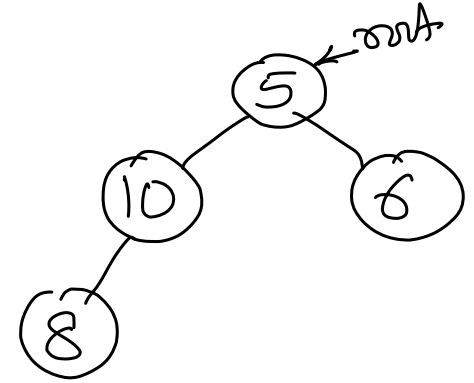
Pre Order (root) **A1**

root
R1

PostOrder(root)

- if (root is empty) then
 - Stop.
- If (root node's left child exists) then
 - PostOrder(root's left child).
- If (root node's right child exists) then
 - PostOrder(root's right child).
- Process root node's data.
- Stop.

O/p: 8 10 6 5

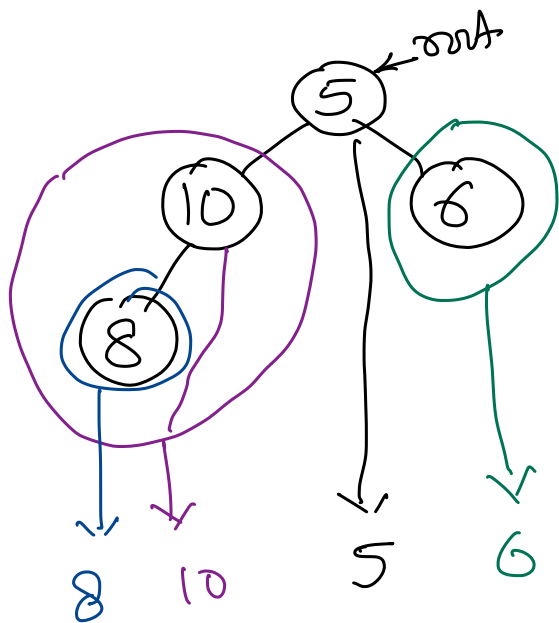


InOrder(root)

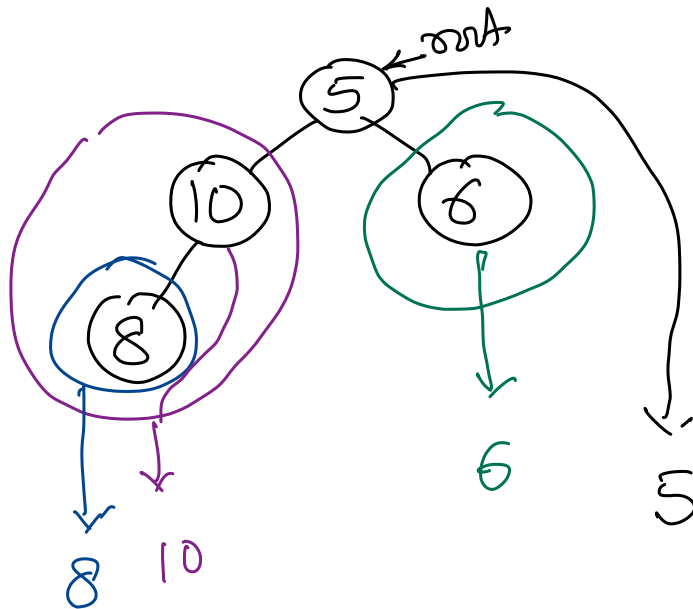
- if (root is empty) then
 - Stop.
- If (root node's left child exists) then
 - InOrder(root's left child).
- Process root node's data.
- If (root node's right child exists) then
 - InOrder(root's right child).
- Stop.

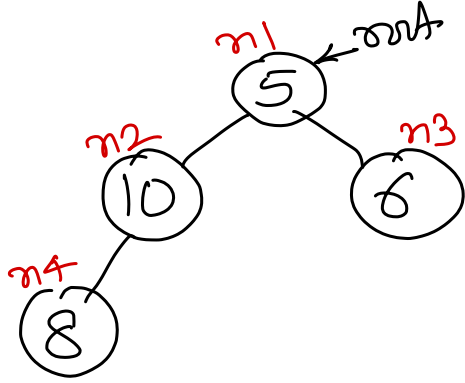
O/p:

In order



Post Order





```

class BTreeNode {

```

```

    int data;

```

```

    BTreeNode leftChild;

```

```

    BTreeNode rightChild;

```

```

}

```

```

BTreeNode n1 = new BTreeNode();

```

```

BTreeNode n2 = new BTreeNode();

```

```

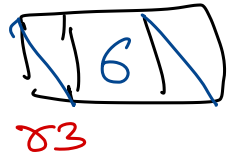
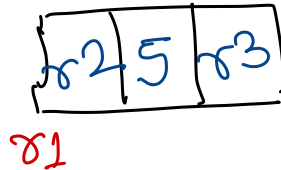
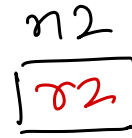
BTreeNode n3 = new BTreeNode();

```

```

BTreeNode n4 = new BTreeNode();

```



n1. data = 5;

n1. left child = n2;

n1. right child = n3

n2. data = 10;

n2. left child = n4;

n2. right child = null;

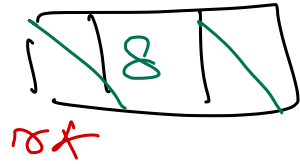
n4. data = 8;

n4. left child = null;

n4. right child = null;

n4

84



n3. data = 6;

n3. left child = null;

n3. right child = null;

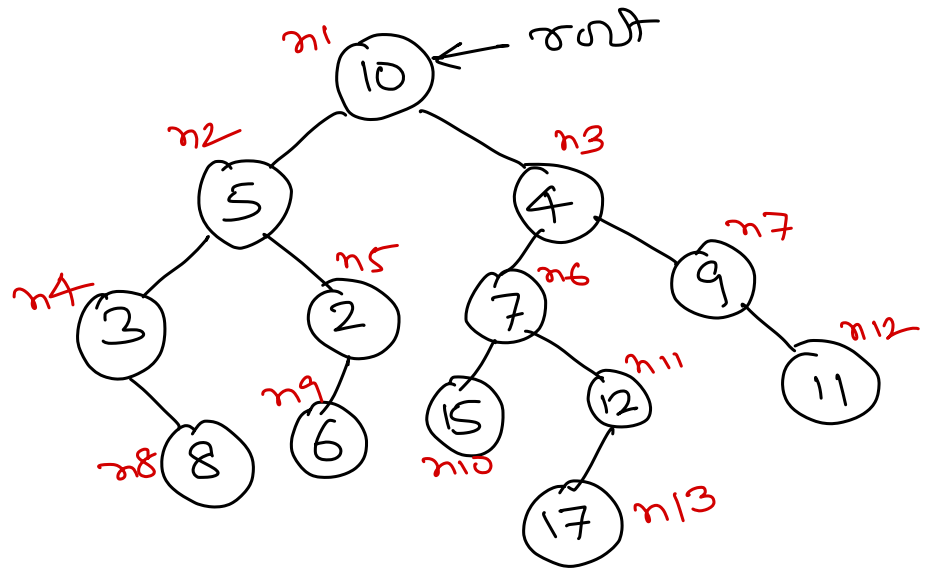
root = n1;

Exercise

→ Find o/p of
Preorder &
Postorder Traversal.

→ Hard code the
tree.

→ Find o/p of all traversals.



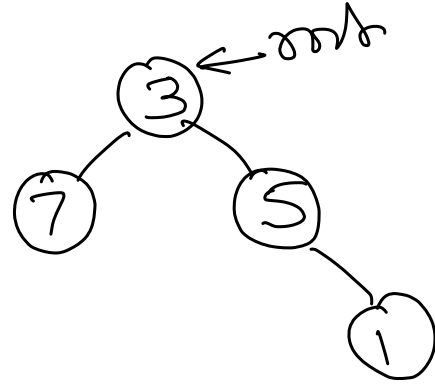
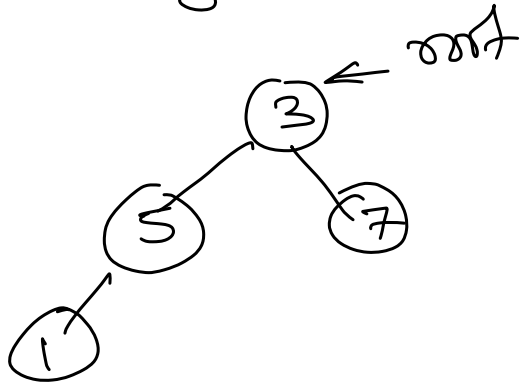
① Count number of nodes in a binary tree.

$$\text{CountNodes}(\text{root}) = \begin{cases} 0, & \text{if root is empty.} \\ 1 + \text{CountNodes}(\text{root's left child}) \\ \quad + \text{CountNodes}(\text{root's right child}) \end{cases}$$

② Count number of leaf nodes in a binary tree.

③ Count frequency of occurrence of a value in binary tree.

④ Invert a binary tree / Find mirror image of binary tree.



⑤ Print using level order but o/p of each level on separate lines.